

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_191a9b86-991\agent-a3d20084060582eef.jsonl`
- SHA-256: `4bbb7351a0650ab2de8253a9aa05b89fed7847e7339902ebe8ae8a7896cdc733`
- Source modified: `2026-06-22T03:49:17+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_191a9b86-991`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T03:45:49.099Z)

Review the P2c wizard for ACCESSIBILITY + UX honesty. Run `cd C:\\Users\\avidu\\Projects\\khelsutra-p2c && git diff origin/master...HEAD` and read web/src/components/SetupWizard.tsx in C:\\Users\\avidu\\Projects\\khelsutra-p2c, comparing against the repo's all conventions (?44px tap targets via min-h-11/h-11, aria-live, role=dialog, visible-not-hover text ? see other components like IngestPanel/StatusChip). Scrutinize: (1) tap targets on ALL wizard buttons (min-h-11)? (2) role=dialog without aria-modal/focus-trap ? is that acceptable for a non-blocking inline panel, or misleading? (3) the external "Get a free Gemini key" link ? target=_blank with rel=noreferrer? (4) does the copy make honest claims (offline motion needs no key; guided key-setup not auto-save)? Report concrete findings (file:line) only; empty list if none.

2. assistant (2026-06-22T03:45:51.410Z)

I'll review the P2c wizard for accessibility and UX honesty. Let me start by examining the diff and the relevant files.

3. user (2026-06-22T03:45:55.173Z)

```
locales/da/common.json      | 32 +++++
locales/en/common.json      | 32 +++++
locales/es/common.json      | 32 +++++
locales/hi/common.json      | 32 +++++
locales/id/common.json      | 32 +++++
locales/kn/common.json      | 32 +++++
web/src/App.tsx              | 9 +-
web/src/api/client.ts        | 6 ++
web/src/api/types.ts         | 17 +++
web/src/components/SetupWizard.test.tsx | 115 +++++
web/src/components/SetupWizard.tsx      | 179 +++++
web/src/components/a4.touch.test.tsx    | 4 +
12 files changed, 521 insertions(+), 1 deletion(-)
```

4. user (2026-06-22T03:45:55.413Z)

feat/p2c-setup-wizard
e6b9725 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
cf59fbf feat(site): add es/da/id to the marketing site (parity with the SPA's 6 languages) (#87)
17242a9 feat(site): localize own-model body content to kn/hi (#86)
7b04f02 deploy(kit): align with the live-verified droplet deploy (+ the gemini-default fix) (#85)
5e5a2ac docs(friendly-multilingual): close out the chapter (\$9 DoD + UI/UX + B7 deferred) (#84)

5. assistant (2026-06-22T03:45:57.721Z)

Let me read the SetupWizard component and the reference components for ally conventions.

6. user (2026-06-22T03:45:58.747Z)

```
1      // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2      // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box
can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "kheisutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15         try {
16             return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17         } catch {
18             return false; // private-mode / no storage ? show it (harmless; user can skip)
19         }
20     }
21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-block text-sm font-medium text-green underline"
41                         >
42                             {t("wizard.key.getKey")}
43                         </a>
44                     </>
45                 )}
46             </div>
47         );
48     }
49
50     function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51         const { t } = useTranslation();
52         const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53         const gpu = caps?.gpu?.nvidia_smi_present === true;
54         const detector = caps?.default_segmenter ?? "motion";
55         return (
56             <div>
```

```

57         <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58         <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59         <ul className="mt-3 space-y-1 text-sm text-ink/70">
60             <li>{t("wizard.compute.detector", { name: detector })}</li>
61             <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62             <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
63         </ul>
64     </div>
65 );
66 }
67
68 function VideosStep() {
69     const { t } = useTranslation();
70     return (
71         <div>
72             <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73             <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74             <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75         </div>
76     );
77 }
78
79 export function SetupWizard({ onClose }: { onClose: () => void }) {
80     const { t } = useTranslation();
81     const [step, setStep] = useState(0);
82     const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83     const [loading, setLoading] = useState(true);
84     const [offline, setOffline] = useState(false);
85
86     useEffect(() => {
87         let alive = true;
88         const cid = newCorrelationId();
89         getCapabilities(cid)
90             .then((c) => {
91                 if (!alive) return;
92                 setCaps(c);
93                 setLoading(false);
94             })
95             .catch((err) => {
96                 if (!alive) return;
97                 // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                 setOffline(true);
99                 setLoading(false);
100                 log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101             });
102         return () => {
103             alive = false;
104         };
105     }, []);
106
107     const finish = () => {
108         try {
109             localStorage.setItem(WIZARD_DONE_KEY, "true");
110         } catch {
111             /* private mode: still close (the wizard just reappears next visit) */
112         }
113         onClose();
114     };
115
116     const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117     const isLast = step === STEPS - 1;
118
119     return (

```

```

120     <section
121       role="dialog"
122       aria-label={t("wizard.title")}
123       className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"
124     >
125       <div className="flex items-center justify-between gap-3">
126         <h2 className="text-xl font-bold">{t("wizard.title")}</h2>
127         <button
128           type="button"
129           onClick={finish}
130           className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131         >
132           {t("wizard.skip")}
133         </button>
134       </div>
135       <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136       {stepping && (
137         <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138       )}
139
140       <div className="mt-5 min-h-[128px]" aria-live="polite">
141         {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142         {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143         {stepping && step === 0 && <KeyStep caps={caps} />}
144         {stepping && step === 1 && <ComputeStep caps={caps} />}
145         {stepping && step === 2 && <VideosStep />}
146       </div>
147
148       <div className="mt-6 flex items-center justify-between">
149         {stepping ? (
150           <>
151             <button
152               type="button"
153               disabled={step === 0}
154               onClick={() => setStep((s) => Math.max(0, s - 1))}
155               className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156             >
157               {t("wizard.back")}
158             </button>
159             <button
160               type="button"
161               onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
162               1)))}
163               className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
164             >
165               {isLast ? t("wizard.done") : t("wizard.next")}
166             </button>
167           </>
168         ) : (
169           <button
170             type="button"
171             onClick={finish}
172             className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
173           >
174             {t("wizard.done")}
175           </button>
176         )}
177       </div>
178     </section>
179   );
180 }

```

7. user (2026-06-22T03:45:59.349Z)

CompileResult.tsx
 IngestPanel.test.tsx
 IngestPanel.tsx
 LanguageSwitcher.tsx
 QueryBar.examples.test.ts
 QueryBar.test.tsx
 QueryBar.tsx
 RallyGrid.tsx
 SelectionFooter.test.tsx
 SelectionFooter.tsx
 SetupWizard.test.tsx
 SetupWizard.tsx
 StatusChip.tsx
 TapFilters.test.tsx
 TapFilters.tsx
 a4.touch.test.tsx
 a8.status.test.tsx
 build-error.test.tsx
 upload.test.tsx

8. assistant (2026-06-22T03:46:02.024Z)

Now let me examine the reference components for a11y conventions to compare against.

9. user (2026-06-22T03:46:03.489Z)

```

1      // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2      // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3      // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4      // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5      // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6      // (done but no rallies) is shown, never silently treated as a load.
7      import { useState } from "react";
8      import type { ChangeEvent, FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11     import { useUpload } from "../hooks/useUpload";
12     import { errorText } from "../lib/errorText";
13     import { StatusChip, Spinner } from "./StatusChip";
14
15     interface Props {
16       /** Called with the engine's video_id once a submitted job finishes successfully. */
17       onLoaded: (videoId: string) => void;
18     }
19
20     export function IngestPanel({ onLoaded }: Props) {
21       const { t, i18n } = useTranslation();
22       const [uri, setUri] = useState("");
23       const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
24       // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
25       const locale = i18n.resolvedLanguage ?? i18n.language;
26       // On upload completion, feed the returned server PATH straight into the engine
processing back-half.

```

```

27     const upload = useUpload((path) => submit(path, locale));
28     const busy = ingestBusy || upload.busy;
29
30     const onSubmitUri = (e: FormEvent) => {
31         e.preventDefault();
32         submit(uri, locale);
33     };
34     const onFile = (e: ChangeEvent<HTMLInputElement>) => {
35         const file = e.target.files?.[0];
36         if (file) upload.start(file);
37         e.target.value = ""; // let re-picking the same file fire change again
38     };
39     const cancelAll = () => {
40         upload.cancel();
41         cancelIngest();
42     };
43     const resetAll = () => {
44         upload.reset();
45         resetIngest();
46     };
47
48     // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
49     // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
50     // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
51     const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
52     const errorMsg = errorState ? errorText(t, errorState) : null;
53
54     return (
55         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
56             <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
57             <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
58
59             {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
60             <div className="mt-4">
61                 <label
62                     className={
63                         // focus-within makes the visible button show a focus ring when the (sr-
only) input is
64                         // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
65                         "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
66                         (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
67                     }
68                 >
69                     {t("ingest.chooseFile")}
70                     <input
71                         type="file"
72                         accept=".mp4,.mov,video/mp4,video/quicktime"
73                         aria-label={t("ingest.uploadAria")}
74                         disabled={busy}
75                         onChange={onFile}
76                         className="sr-only"
77                     />
78                 </label>
79                 <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
80             </div>
81
82             {/* SECONDARY: or point at a cloud bucket / host path. */}

```

```

83         <div className="mt-4 border-t border-black/5 pt-4">
84             <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
85             <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
86                 <input
87                     type="text"
88                     value={uri}
89                     onChange={(e) => setUri(e.target.value)}
90                     placeholder={t("ingest.uriPlaceholder")}
91                     aria-label={t("ingest.uriAriaLabel")}
92                     disabled={busy}
93                     className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
94                 />
95                 <button
96                     type="submit"
97                     disabled={busy}
98                     className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
99                 >
100                     {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
101                 </button>
102             </form>
103             <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
104         </div>
105
106         {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
107         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
108         <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
109             {upload.phase.status === "uploading" && (
110                 <div data-testid="upload-progress">
111                     {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
112                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
113                     <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
114                     <div className="mt-2 flex max-w-sm items-center gap-2">
115                         <div
116                             className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
117                             role="progressbar"
118                             aria-label={t("ingest.uploadProgress")}
119                             aria-valuenow={upload.phase.pct}
120                             aria-valuemin={0}
121                             aria-valuemax={100}
122                         >
123                             <div className="h-full bg-green transition-all" style={{ width:
`${upload.phase.pct}%` }} />
124                         </div>
125                         <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
{upload.phase.pct}%
126                         </span>
127                     </div>
128                     <div>
129                         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
130                         <button
131                             type="button"
132                             onClick={cancelAll}
133                             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
134                         >
135                             {t("ingest.cancel")}
136                         </button>

```

```

137         </div>
138     )}
139     {ingestBusy && (
140         <div>
141             <StatusChip
142                 tone="info"
143                 icon={<Spinner />}
144                 label={
145                     t("ingest.working") +
146                     (phase.status === "polling" && phase.progress
147                         ? ` · ${t("ingest.progress", { progress: phase.progress })}`
148                         : "")
149                 }
150             />
151             {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
152             <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
153             <button
154                 type="button"
155                 onClick={cancelAll}
156                 className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
157                 >
158                 {t("ingest.cancel")}
159             </button>
160         </div>
161     )}
162     {errorMsg && (
163         <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
164             <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
165             <p className="mt-2">{errorMsg}</p>
166             <button
167                 type="button"
168                 onClick={resetAll}
169                 className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
170                 >
171                 {t("ingest.retry")}
172             </button>
173         </div>
174     )}
175 </div>
176 </section>
177 );
178 }
179

```

10. user (2026-06-22T03:46:03.932Z)

```

1      // A status indicator that is ALWAYS icon + text ? never icon-only (HCI4D guidance for
non-tech-savvy
2      // users, FRIENDLY_AND_MULTILINGUAL §4: pair every icon with a word, don't rely on color
alone). Used
3      // for the ingest "working / couldn't process" states and the build "stitching / ready /
failed" states.
4      import type { ReactNode } from "react";
5
6      type Tone = "info" | "error";
7
8      const TONE: Record<Tone, string> = {
9          info: "bg-ink/5 text-ink/70",
10         error: "bg-red-100 text-red-800",
11     };
12

```



```

13     interface Props {
14         /** A glyph or the <Spinner/> ? decorative; the label carries the meaning for screen
readers. */
15         icon: ReactNode;
16         label: string;
17         tone?: Tone;
18     }
19
20     export function StatusChip({ icon, label, tone = "info" }: Props) {
21         return (
22             <span className={`inline-flex items-center gap-1.5 rounded-full px-3 py-1 text-sm
font-medium ${TONE[tone]}`}>
23                 <span aria-hidden="true" className="inline-flex">
24                     {icon}
25                 </span>
26                 <span>{label}</span>
27             </span>
28         );
29     }
30
31     /** Indeterminate spinner ? the engine reports coarse phase WORDS, not a percentage, so
progress is a
32     * spinner + the phase label rather than a misleading progress bar. */
33     export function Spinner() {
34         return (
35             <span
36                 className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
37                 aria-hidden="true"
38             />
39         );
40     }
41

```

11. user (2026-06-22T03:46:04.335Z)

```

1     // @vitest-environment jsdom
2     //
3     // A4 ? non-tech-savvy mobile UX. Two guarantees a phone user depends on:
4     // 1. TOUCH TARGETS ? 44px (WCAG 2.5.5 / platform HIG) on every primary control ?
pinned via the
5     // Tailwind min-h-11 (=2.75rem=44px) / h-11 class so a future "tighten the padding"
can't
6     // silently shrink a tap target below the floor.
7     // 2. The "not available yet" REASON is VISIBLE text, not a hover-only title ? a touch
device never
8     // surfaces `title=`, so the explanation must render in the DOM for a phone user to
read it.
9     import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
10    import { render, screen, cleanup, fireEvent } from "@testing-library/react";
11    import i18n from "../i18n/index";
12    import App from "../App";
13    import { QueryBar } from "../QueryBar";
14    import { WIZARD_DONE_KEY } from "../SetupWizard";
15
16    // 44px floor: min-h-11 (buttons/inputs) or an explicit h-11 square (the ? dismiss).
17    const TAP_44 = /\b(?:min-h-11|h-11)\b/;
18
19    beforeEach(async () => {
20        vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
21        window.localStorage.clear();
22        // Skip the first-run wizard (P2c) so these tests render the app proper (IngestPanel +
grid) whose
23        // touch targets they assert ? the wizard's own ?44px targets are covered in
SetupWizard.test.tsx.

```

```

24     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
25     await i18n.changeLanguage("en");
26 });
27
28 afterEach(() => {
29     cleanup();
30     vi.unstubAllGlobals();
31 });
32
33 describe("A4: touch targets ? 44px on primary controls", () => {
34     it("the always-visible buttons carry a ?44px tap class", () => {
35         render(<App />);
36         for (const name of [
37             "Select all",
38             "Clear",
39             "Invert",
40             "Apply", // QueryBar submit
41             "Process video", // IngestPanel submit
42             "English", // LanguageSwitcher
43             "?????",
44             "??????",
45         ]) {
46             const btn = screen.getByRole("button", { name });
47             expect(btn.className, `${name} must be ?44px`).toMatch(TAP_44);
48         }
49         // The Build button label includes the live selection count ("Build reel (6)").
50         expect(screen.getByRole("button", { name: /Build reel/
51         })).className).toMatch(TAP_44);
52     });
53
54     it("the text inputs carry a ?44px tap class", () => {
55         render(<App />);
56         expect(screen.getByRole("textinput", { name: "Filter rallies by length, count or
57         order" })).className).toMatch(TAP_44);
58         expect(screen.getByRole("textinput", { name: "Reel name"
59         })).className).toMatch(TAP_44);
60         expect(
61             screen.getByRole("textinput", { name: "Video location ? a gs:// bucket URL or a path
62             on the engine host" })).className,
63         ).toMatch(TAP_44);
64     });
65
66     it("the sample-query chips are tappable (?44px)", () => {
67         render(<App />);
68         expect(screen.getByRole("button", { name: "over 10s" })).className).toMatch(TAP_44);
69         expect(screen.getByRole("button", { name: "top 15 longest"
70         })).className).toMatch(TAP_44);
71     });
72
73     describe("A4: per-card shot-count confidence is visible, not hover-only", () => {
74         it("renders the confidence inline on the chip (phones never see a title=)", () => {
75             const { container } = render(<App />); // demo rallies include a High-confidence
76             card
77             expect(container.textContent).toContain("confidence: High");
78         });
79     });
80
81     describe("A4: the 'not available yet' reason is visible, not hover-only", () => {
82         const HELP = "Shot type, players and score aren't available yet ? filter by rally
83         length instead.";
84
85         it("hidden until an unsupported concept is typed, then shown as plain text", () => {
86             render(<QueryBar onApply={() => {}} />);
87             expect(screen.queryByText(HELP)).toBeNull(); // nothing typed ? no help line

```

```

82         fireEvent.change(screen.getByRole("textbox", { name: "Filter rallies by length,
count or order" })), {
83             target: { value: "smashes" }, // a shot-type token ? flagged "not available yet"
84         });
85         expect(screen.getByText(HELP)).toBeTruthy(); // the reason is now a visible DOM node
86     });
87 });
88

```

12. assistant (2026-06-22T03:46:07.075Z)

Let me check the SetupWizard test file and the App.tsx integration, plus the en locale strings for the copy honesty review.

13. user (2026-06-22T03:46:08.013Z)

```

1      // @vitest-environment jsdom
2      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
3      import { render, screen, fireEvent, cleanup, waitFor } from "@testing-library/react";
4      import i18n from "../i18n/index";
5      import { SetupWizard, WIZARD_DONE_KEY, isWizardDone } from "../SetupWizard";
6      import type { EngineCapabilities } from "../api/types";
7
8      function stubCapabilities(caps: EngineCapabilities | "reject") {
9          vi.stubGlobal(
10              "fetch",
11              vi.fn(async (url: string | URL) => {
12                  const u = typeof url === "string" ? url : url.toString();
13                  if (u.endsWith("/api/capabilities")) {
14                      if (caps === "reject") throw new TypeError("Failed to fetch");
15                      return { ok: true, status: 200, json: async () => caps } as unknown as Response;
16                  }
17                  return { ok: false, status: 404, text: async () => "" } as unknown as Response;
18              }),
19          );
20      }
21
22      const LIGHT: EngineCapabilities = {
23          gemini_key_present: false,
24          default_segmenter: "motion",
25          ffmpegpeg: { ffmpegpeg: false, ffprobe: false },
26          gpu: { nvidia_smi_present: false },
27      };
28      const READY: EngineCapabilities = {
29          gemini_key_present: true,
30          default_segmenter: "gemini",
31          ffmpegpeg: { ffmpegpeg: true, ffprobe: true },
32          gpu: { nvidia_smi_present: true },
33      };
34
35      beforeEach(async () => {
36          window.localStorage.clear();
37          await i18n.changeLanguage("en");
38      });
39      afterEach(() => {
40          cleanup();
41          vi.restoreAllMocks();
42          vi.unstubAllGlobals();
43      });
44
45      describe("SetupWizard", () => {
46          it("checks capabilities, then step 1 guides a missing Gemini key (offline-first
default)", async () => {
47              stubCapabilities(LIGHT);
48              render(<SetupWizard onClose={() => {}} />);

```

```

49     expect(screen.getByText(/Checking what this device can do/)).toBeTruthy();
50     expect(await screen.findByText(/built-in motion detector/)).toBeTruthy();
51     expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
52     expect(screen.getByText(/Step 1 of 3/)).toBeTruthy();
53 });
54
55     it("walks all 3 steps from real capabilities; Back returns; Done sets the flag +
closes", async () => {
56         stubCapabilities(LIGHT);
57         const onClose = vi.fn();
58         render(<SetupWizard onClose={onClose} />);
59         await screen.findByText(/built-in motion detector/);
60
61         fireEvent.click(screen.getByText("Next")); // ? compute
62         expect(screen.getByText("Detector: motion")).toBeTruthy();
63         expect(screen.getByText(/ffmpeg not found/)).toBeTruthy();
64         expect(screen.getByText(/No GPU needed/)).toBeTruthy();
65
66         fireEvent.click(screen.getByText("Back")); // ? key
67         expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
68
69         fireEvent.click(screen.getByText("Next")); // ? compute
70         fireEvent.click(screen.getByText("Next")); // ? videos
71         expect(screen.getByText(/You're all set/)).toBeTruthy();
72
73         fireEvent.click(screen.getByText("Get started")); // Done on last step
74         expect(onClose).toHaveBeenCalledTimes(1);
75         expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
76     });
77
78     it("reflects a connected key + a GPU/ffmpeg-ready box", async () => {
79         stubCapabilities(READY);
80         render(<SetupWizard onClose={() => {}} />);
81         expect(await screen.findByText(/Gemini is connected/)).toBeTruthy();
82         fireEvent.click(screen.getByText("Next"));
83         expect(screen.getByText("Detector: gemini")).toBeTruthy();
84         expect(screen.getByText(/Video tools \ffmpeg\ ready/)).toBeTruthy();
85         expect(screen.getByText(/A GPU was detected/)).toBeTruthy();
86     });
87
88     it("Skip closes + sets the flag without finishing the steps", async () => {
89         stubCapabilities(LIGHT);
90         const onClose = vi.fn();
91         render(<SetupWizard onClose={onClose} />);
92         await screen.findByText(/built-in motion detector/);
93         fireEvent.click(screen.getByText("Skip"));
94         expect(onClose).toHaveBeenCalledTimes(1);
95         expect(isWizardDone()).toBe(true);
96     });
97
98     it("degrades gracefully when the engine is unreachable (offline) ? message + a Done
escape", async () => {
99         stubCapabilities("reject");
100        const onClose = vi.fn();
101        render(<SetupWizard onClose={onClose} />);
102        expect(await screen.findByText(/Couldn't reach KhelSutra/)).toBeTruthy();
103        // No step content / step indicator when offline.
104        expect(screen.queryByText(/Step 1 of 3/)).toBeNull();
105        fireEvent.click(screen.getByText("Get started"));
106        await waitFor(() => expect(onClose).toHaveBeenCalledTimes());
107        expect(isWizardDone()).toBe(true);
108    });
109
110    it("isWizardDone reads the localStorage flag", () => {
111        expect(isWizardDone()).toBe(false);

```

```

112         window.localStorage.setItem(WIZARD_DONE_KEY, "true");
113         expect(isWizardDone()).toBe(true);
114     });
115 });
116

```

14. user (2026-06-22T03:46:09.296Z)

```

diff --git a/locales/en/common.json b/locales/en/common.json
index aaaa01a..45bc8be 100644
--- a/locales/en/common.json
+++ b/locales/en/common.json
@@ -288,6 +288,38 @@
     }
   },
+  "wizard": {
+    "title": "Welcome to KhelSutra",
+    "intro": "Three quick steps and you're ready to turn a match into a highlights reel.",
+    "step": "Step {n} of 3",
+    "checking": "Checking what this device can do?",
+    "offline": "Couldn't reach KhelSutra ? start it, then reopen setup. You can skip for now.",
+    "back": "Back",
+    "next": "Next",
+    "skip": "Skip",
+    "done": "Get started",
+    "key": {
+      "title": "AI highlights (optional)",
+      "connected": "Gemini is connected ? you'll get AI-assisted highlights.",
+      "missing": "No AI key yet ? that's fine. KhelSutra works offline with the built-in motion detector.",
+      "how": "Want sharper highlights? Get a free Gemini key, add GEMINI_API_KEY=? to the .env file in your KhelSutra folder, then restart.",
+      "getKey": "Get a free Gemini key"
+    },
+    "compute": {
+      "title": "Where it runs",
+      "local": "Everything runs on this device ? your videos never leave it.",
+      "detector": "Detector: {name}",
+      "ffmpegOk": "Video tools (ffmpeg) ready.",
+      "ffmpegMissing": "ffmpeg not found ? install it so KhelSutra can cut and join clips.",
+      "gpu": "A GPU was detected ? the high-quality shuttle tracker can run here later.",
+      "noGpu": "No GPU needed ? the offline motion detector runs on any computer."
+    },
+    "videos": {
+      "title": "Add a video",
+      "body": "When you're ready, pick a video file or paste a cloud link and KhelSutra finds the rallies for you.",
+      "ready": "You're all set."
+    }
+  },
+  "selection": {
+    "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
+  }
}

```

15. assistant (2026-06-22T03:46:11.923Z)

Now let me check how the wizard is integrated in App.tsx and look at the LanguageSwitcher for the `\_blank`/`rel` convention and any other external links in the codebase.

16. user (2026-06-22T03:46:13.553Z)

```

diff --git a/web/src/App.tsx b/web/src/App.tsx
index a0a7882..c516153 100644

```

```

--- a/web/src/App.tsx
+++ b/web/src/App.tsx
@@ -8,6 +8,7 @@ import { SelectionFooter } from "../components/SelectionFooter";
import { LanguageSwitcher } from "../components/LanguageSwitcher";
import { CompileResult, type BuildState } from "../components/CompileResult";
import { IngestPanel } from "../components/IngestPanel";
+import { SetupWizard, isWizardDone } from "../components/SetupWizard";
import { runQuery, type ParsedQuery } from "../lib/query";
import { floorRisk } from "../lib/floor";
import { reelFilename } from "../lib/format";
@@ -60,6 +61,8 @@ export default function App() {
  const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" } : {
status: "demo" });
  const [reelName, setReelName] = useState("highlights");
  const [build, setBuild] = useState<BuildState>({ status: "idle" });
+ // P2c: the first-run wizard shows once per browser, before the ingest panel, when nothing's
loaded.
+ const [wizardDone, setWizardDone] = useState(isWizardDone);

  // Display order is separate from selection: a query can re-sort the cards while the shared
  // selection set (sel) stays the single source of truth for what lands in the reel.
@@ -234,9 +237,13 @@ export default function App() {
  <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
  </p>

+    { /* P2c first-run wizard: before anything is loaded, walk a new user through AI-key /
compute /
+      videos (from /api/capabilities). Shows once per browser; then the ingest panel
takes over. */ }
+    { !videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} /> }
+
    { /* Front-half ingest (B-P2): when no video is loaded, point the tool at a bucket URI /
host
      path ? process ? load. Once a video_id exists, the existing back-half takes over.
*/ }
-    { !videoId && (
+    { !videoId && wizardDone && (
      <IngestPanel
        onLoad={() => {
          setVideoId(id);

```

17. user (2026-06-22T03:46:14.865Z)

```

C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\CompileResult.tsx-79-
href={ `https://wa.me/?text=${encodeURIComponent(t("result.shareText"))}` }
C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\CompileResult.tsx:80:
target="_blank"
C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\CompileResult.tsx-81-
rel="noopener noreferrer"
--
C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\SetupWizard.tsx-37-
href="https://aistudio.google.com/apikey"
C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\SetupWizard.tsx:38:
target="_blank"
C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\SetupWizard.tsx-39-
rel="noreferrer"

```

18. assistant (2026-06-22T03:46:19.078Z)

I found a key inconsistency: the existing external link in `CompileResult` uses ``rel="noopener noreferrer"``, but the new wizard link uses ``rel="noreferrer"``. Let me verify the ``noreferrer``/``noopener`` behavior and check the `App.tsx` wizard wrapper for ``role=dialog`` context. Let me also confirm whether there's a focus-management/keyboard pattern used elsewhere and the ``min-h-11`` floor on the external link.

Let me look at the capabilities endpoint to confirm the copy's honesty claims about `gemini\_key\_present` and the key-write absence.

19. user (2026-06-22T03:46:20.725Z)

```
diff --git a/web/src/api/client.ts b/web/src/api/client.ts
index fa74d2f..23c8dd4 100644
--- a/web/src/api/client.ts
+++ b/web/src/api/client.ts
@@ -7,6 +7,7 @@ import type {
  QueryResponse,
  CompileResponse,
  EngineConfig,
+ EngineCapabilities,
  ProcessResponse,
  JobStatus,
  UploadInitResponse,
@@ -133,6 +134,11 @@ export async function getConfig(cid?: string): Promise<EngineConfig> {
  return request<EngineConfig>("/config", { method: "GET", cid });
}

+/** What THIS box can do ? drives the first-run wizard. GET /api/capabilities (PACKAGING_PLAN
P2c). */
+export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
+ return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
+}
+
+/** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
export function minShotCount(cfg: EngineConfig): number | undefined {
  return cfg.stitching?.min_shot_count;
}
diff --git a/web/src/api/types.ts b/web/src/api/types.ts
index 4978d7d..59749ae 100644
--- a/web/src/api/types.ts
+++ b/web/src/api/types.ts
@@ -45,6 +45,23 @@ export interface EngineConfig {
  [key: string]: unknown;
}

+/** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do, rendered
by
+ * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini key
value).
+ * Every field optional/defensive: a missing field degrades the wizard gracefully. */
+export interface EngineCapabilities {
+ api_version?: number;
+ engine_version?: string;
+ segmenters?: string[];
+ default_segmenter?: string;
+ ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?: string
};
+ gpu?: { nvidia_smi_present?: boolean };
+ /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step. */
+ gemini_key_present?: boolean;
+ wasb_weights_present?: boolean;
+ deployment?: { profile?: string; compute_target?: string };
+ [key: string]: unknown;
+}
+
+/** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
export interface ProcessResponse {
  job_id: string;
```

20. user (2026-06-22T03:46:21.795Z)

No files found

21. assistant (2026-06-22T03:46:27.473Z)

The backend isn't in this repo (engine is a sibling). The copy claim "the engine has no key-write endpoint yet, so we tell the user how to add the key" matches the guided (not auto-save) UX ? and there is no save action in the component, consistent with the comment. Let me verify the i18n interpolation for `wizard.compute.detector` uses `{name}` and check the `text-amber-700` color token usage (since other components use specific tokens). Let me also confirm there's no `aria-modal` and check focus behavior.

Let me verify the `min-h-11` floor specifically on the external "Get key" link ? it's an `` not a button, so it's not asserted in the A4 test. Let me check the test coverage and color token consistency.

22. user (2026-06-22T03:46:29.508Z)

```
C:\Users\avidu\Projects\khelsutra-p2c\web\src\index.css:4:   from these ? e.g. bg-paper, text-ink, bg-ink, text-lime, text-green. */
C:\Users\avidu\Projects\khelsutra-p2c\web\src\App.tsx:225:           Khel<span className="text-green">?</span>Sutra <span className="font-semibold text-lime">Studio</span>
C:\Users\avidu\Projects\khelsutra-p2c\web\src\App.tsx:232:           <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-semibold text-green">
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\CompileResult.tsx:45:           <span aria-hidden="true" className={isError ? "text-red-700" : "text-green"}>{isError ? "?" : "?"}</span>
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\CompileResult.tsx:83:           className="inline-flex min-h-11 items-center gap-1.5 rounded-full border border-green/50 px-4 py-2 text-sm font-semibold text-green hover:bg-green/5"
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\QueryBar.tsx:68:           <span key={c} className={CHIP + " bg-green/15 text-green"}>
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\QueryBar.tsx:110:           <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60" />, x: <span className="text-amber-700" /> }} />
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\RallyGrid.tsx:65:           className="mt-2 inline-block rounded-full bg-amber-50 px-2 py-0.5 text-xs text-amber-700"
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SelectionFooter.tsx:35:           : "border-amber-200 bg-amber-50 text-amber-900")
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SetupWizard.tsx:31:           <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SetupWizard.tsx:40:           className="mt-2 inline-block text-sm font-medium text-green underline"
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SetupWizard.tsx:74:           <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SetupWizard.tsx:142:           {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
```

23. user (2026-06-22T03:48:27.103Z)

<persisted-output>

Output too large (336.1KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\tool-results\beizsl8bq.txt

Preview (first 2KB):

```
web/dist/assets/index-BcE6Y0gS.css:1:@font-face{font-family:Noto Sans Kannada Variable;font-style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans-kannada-kannada-wght-normal-DuWB-P8d.woff2) format("woff2-variations");unicode-range:U+0951-0952,U+0964-0965,U+0C80-0CF3,U+1CD0,U+1CD2-1CD3,U+1CDA,U+1CF2,U+1CF4,U+200C-200D,U+20B9,U+25CC,U+A830-A835}@font-face{font-family:Noto Sans Kannada Variable;font-style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2) format("woff2-variations");unicode-range:U+0100-02BA,U+02BD-02C5,U+02C7-02CC,U+02CE-02D7,U+02DD-02FF,U+0304,U+0308,U+0329,U+1D00-1DBF,U+1E00-1E9F,U+1EF2-1EFF,U+2020,U+20A0-20AB,U+20AD-20C0,U+2113,U+2C60-2C7F,U+A720-A7FF}@font-face{font-family:Noto Sans Kannada Variable;font-
```



```

style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2) format("woff2-variations");unicode-range:U+0000-00FF,U+0131,U+0152-0153,U+02BB-02BC,U+02C6,U+02DA,U+02DC,U+0304,U+0308,U+0329,U+2000-206F,U+20AC,U+2122,U+2191,U+2193,U+2212,U+2215,U+FEFF,U+FFFD}@font-face{font-family:Noto Sans Devanagari Variable;font-style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2) format("woff2-variations");unicode-range:U+0900-097F,U+1CD0-1CF9,U+200C-200D,U+20A8,U+20B9,U+20F0,U+25CC,U+A830-A839,U+A8E0-A8FF,U+11B00-11B09}@font-face{font-family:Noto Sans Devanagari Variable;font-style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans-devanagari-latin-ext-wght-normal-CLkUpaxV.woff2) format("woff2-variations");unicode-range:U+0100-02BA,U+02BD-02C5,U+02C7-02CC,U+02CE-02D7,U+02DD-02FF,U+0304,U+0308,U+0329,U+1D00-1DBF,U+1E00-1E9F,U+1EF2-1EFF,U+2020,U+20A0-20AB,U+20AD-20C0,U+2113,U+2C60-2C7F,U+A720-A7FF}@font-face{font-family:Noto Sans Devanagari Variable;font-style:normal;font-display:swap;font-weight:100 900;src:url(/assets/noto-sans
...
</persisted-output>

```

24. assistant (2026-06-22T03:48:41.126Z)

The `text-amber-700` token is consistent with other components (QueryBar, RallyGrid). The `text-green` color token is also consistent. Now I have everything needed. Let me verify the key honesty points and the tap-target/dialog semantics.

Summary of my analysis against the four scrutiny points:

(1) Tap targets ?44px: All wizard ``s carry `min-h-11` (Skip line 130, Back line 155, Next/Done line 162, offline Done line 171). However, the external "Get a free Gemini key" link (`<a>`, lines 36-43) carries only `text-sm ... inline-block` ? no `min-h-11`. It's an interactive control that fails the ?44px floor the repo pins via A4. It's also not covered by the A4 touch test (which only checks `role="button"` and textboxes).

(2) `role="dialog"` without `aria-modal`/focus-trap: The section is `role="dialog"` (line 121) but it's a non-blocking inline panel (rendered inline in the page flow, page remains scrollable/interactive behind it ? actually it replaces IngestPanel, but the header/nav/language switcher remain interactive). It has no `aria-modal`, no focus trap, no initial focus move, no Escape handler. For a true modal dialog this would be misleading, but since it's explicitly a non-blocking inline panel, `role="dialog"` itself sets a modal expectation that isn't met. The more honest role for an inline, non-trapping panel would be `role="region"` (with `aria-label`), or it should commit to modal semantics.

(3) External link `rel`: Line 39 uses `rel="noreferrer"`. `noreferrer` does imply `noopener` in modern browsers, so it's secure. But the sibling external link in CompileResult.tsx:81 uses `rel="noopener noreferrer"` ? an inconsistency, and the explicit `noopener` is the repo convention.

(4) Copy honesty: Strong. "works offline with the built-in motion detector" (no key needed) is accurate; the key step is guided ("add GEMINI_API_KEY=? to the .env file ? then restart") not auto-save ? matching the absence of any save action. "your videos never leave it" matches local compute. Honest.

Let me confirm the App.tsx renders the wizard inline (not as an overlay) to nail down the dialog-semantics finding.

25. user (2026-06-22T03:48:42.298Z)

```

218     };
219
220     return (
221       <main className="min-h-screen bg-paper text-ink pb-24">
222         <header className="bg-ink text-white">
223           <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
224             <span className="text-xl font-extrabold tracking-tight">
225               Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>

```

```

226         </span>
227         <LanguageSwitcher />
228     </div>
229 </header>
230
231     <section className="mx-auto max-w-5xl px-6 py-10">
232         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
233             {t("app.badge")}
234         </span>
235         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
236         <p className="mt-2 max-w-2xl text-ink/70">
237             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
238         </p>
239
240         {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
241             videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
242         {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
243
244         {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
245             path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
246         {!videoId && wizardDone && (
247             <IngestPanel
248                 onLoad={id => {
249                     setVideoId(id);
250                     window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
251                 }}
252             />
253         )}
254
255         <ModeBanner load={loadState} videoId={videoId} />
256
257         <div className="mt-6">

```

26. assistant (2026-06-22T03:48:46.319Z)

Confirmed. The wizard renders inline within the page section (line 242) ? not as a focus-trapping overlay. The header (with LanguageSwitcher) above it stays fully interactive, and content below (ModeBanner, etc.) is still in the DOM and reachable. So `role="dialog"` is a misleading label for what is genuinely a non-blocking inline panel.

I have all findings. Let me submit.

27. user (2026-06-22T03:49:07.890Z)

Structured output provided successfully

28. assistant (2026-06-22T03:49:17.100Z)

I reviewed the P2c wizard (`web/src/components/SetupWizard.tsx`) against the repo's a11y conventions and submitted my findings. Summary:

****Two major findings:****

1. ****`SetupWizard.tsx:36-43`**** ? The external "Get a free Gemini key" link (an `` is missing the `min-h-11` tap-target floor that every wizard button correctly carries. It's a real interactive control on the most-used step (fresh user, no key), and the A4 touch test doesn't catch it because that test only queries `role="button"` and textboxes. CompileResult's external link, by contrast, is a `min-h-11` pill.

2. **``SetupWizard.tsx:120-124``** ? ``role="dialog"``` is the **misleading** choice here (the inverse of how the question framed it). The panel renders inline in the page flow (App.tsx:242), the header/LanguageSwitcher stay interactive, and there's no ``aria-modal`/focus-trap/Escape`. For a genuinely non-blocking inline panel, ``role="region"``` (keeping the existing ``aria-label``) is the honest label; ``role="dialog"``` promises modal focus-capture that isn't delivered.

****One minor:**** ``rel="noreferrer"``` (line 39) is functionally safe but diverges from the repo convention ``rel="noopener noreferrer"``` used at CompileResult.tsx:81.

****Copy honesty (nit, no change needed):**** All four claims hold up ? offline motion explicitly needs no key, the key step is guided (no save action in ``KeyStep``, matches the no-key-write-endpoint comment), "your videos never leave it" matches local compute, and "Gemini is connected" is gated on the engine's ``gemini_key_present`` bool (the key value is never exposed in ``types.ts``).